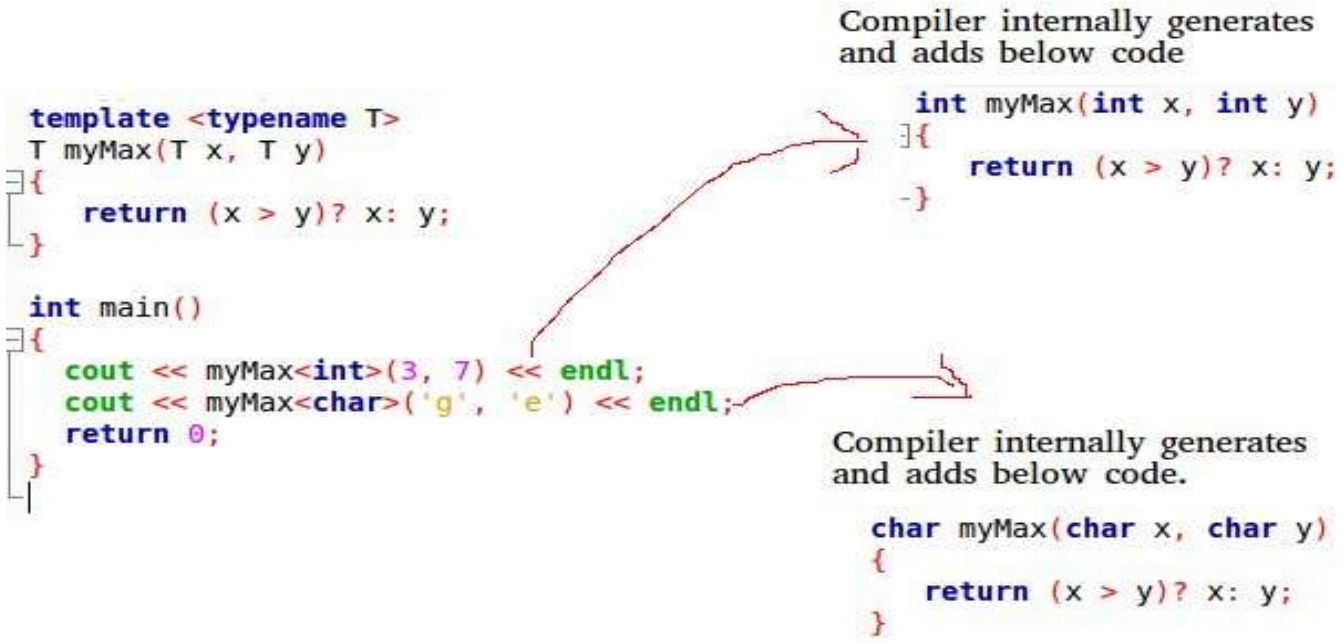
# C++

TEMPLATES, STL, RTTI, SMART POINTER

BY: JANHAVI DEO

# Templates


template <typename T>
T myMax(T x, T y)
{
return (x > y)? x: y;
}
int main()
{
cout << myMax<int>(3, 7) << endl;
cout << myMax<char>('g', 'e') << endl;
return 0;
}
Compiler internally generates and adds below code
int myMax(int x, int y)
{
return (x > y)? x: y;
}
Compiler internally generates and adds below code.
char myMax(char x, char y)
{
return (x > y)? x: y;
}

# Templates

- With function overloading same code needs to be repeated for different data types which leads towards waste of time & space.
- Templates enable us to define generic functions or classes which avoids above repetition of code for different data types .
- Generally, templates are used if same algorithm works well for various data types e.g. sorting algorithms.
- There can be function templates or class templates.
- Function templates can be overloaded.

You can define more than one generic data type in the **template** statement by using a comma-separated list.

```
#include <iostream>
using namespace std;

template <class type1, class type2>
void myfunc(type1 x, type2 y) {
        cout << x << ' ' << y << '\n';
}

int main() {

myfunc(10, "I like C++");
myfunc(98.6, 19L);

return 0;
}
```

# STL (Standard Template Library)

- Provides general-purpose classes and functions that implement many popular and commonly used algorithms and data structures, for example, support for vectors, lists, queues, and stacks.
- Because the STL is constructed from template classes, the algorithms and data structures can be applied to nearly any type of data.
- At the core of the standard template library are four items: *containers, algorithms, functors* and *iterators.*
- STL (Standard Template Library) is basically a **toolbox of ready-made data structures and algorithms.**

# **STL Containers**—the backbone of the Standard Template Library.

Definition: A container is a data structure class in STL that stores collections of objects.

Analogy: Think of containers as different types of storage boxes—some are shelves (vectors), some are lockers (maps), some are queues (lines of people).

Purpose: To abstract away the complexity of memory management and provide ready-made, efficient ways to store and access data.

# STL Containers

Sequence Containers: The main sequence containers are vector, deque, list, forward_list, and array. They store elements in a strict linear order and differ in memory layout, access speed, and insertion/deletion efficiency.

Associative Containers: They organize data based on keys rather than positions. They are smart organizers. They use balanced binary trees or hash tables.

# STL Containers

Unordered Containers: Unlike set or map (which keep elements sorted), unordered containers use hash tables.
This means average O(1) lookup, insertion, and deletion.
Order of elements is not guaranteed — they're arranged by hash values, not by sorting.

Container Adaptors: Instead of creating a data structure from scratch, adaptors wrap around standard sequence containers like std::vector, std::list, or std::deque. They hide unwanted functions and expose only the specific operations needed for a particular data structure

# STL Containers

| Category | Examples | Analogy | Key Use Case |
|---|---|---|---|
| Sequence Containers | vector, list, deque, forward_list, array | Expandable shelf, train couches, two way street, one-way list | Store data in linear order |
| Associative Containers | set, multiset, map, multimap | Unique elements,music playlist, phonebook(one name – one phonenumber), library catalog(one author – multiple books) | Fast lookup, sorted data |
| Unordered Containers | unordered_set, unordered_multiset unordered_map, unordered_multimap | Bunch of marbles (unique but not ordered), bunch of coins, key – value (with unique keys) and key – value ( with duplicate keys) | O(1) average lookup |
| Container Adaptors | stack, queue, priority_queue | FILO, FIFO, sorted queue | Restricted access patterns |

# **STL Containers**—Why to Use

Save time: No need to reinvent data structures.

Efficient: Optimized for speed and memory.

Universal: Work seamlessly with iterators and algorithms.

# Iterators

*Iterators* are objects that are, similar to pointers.

It is like a pointer-like object that helps you move through elements of a container (vector, list, set, map, etc.).

You can increment and decrement Iterators.

You can apply the * operator to them. Iterators are declared using the **iterator** type defined by the various containers.

Provides a uniform way to access elements across different STL containers without worrying about their internal structure.

# Types of Iterators

Types of Iterators :

Input Iterator → Reads data.

Output Iterator → Writes data.

Forward Iterator → Moves forward only.

Bidirectional Iterator → Moves forward and backward.

Random Access Iterator → Jumps anywhere instantly.

# Iterators

Common Operations

begin() → Points to the first element.

end() → Points just past the last element.

++it → Move to next element.

--it → Move to previous element (if supported).

*it → Access the element at iterator's position.

# STL – Vector in C++

**vector** is a dynamic array provided by the STL (Standard Template Library).

A vector can **grow or shrink** automatically during execution.

```
vector<int> v;
v.push_back(10);
v.push_back(20);
v.push_back(30);
cout << v[0];      // first element
cout << v.at(1);   // second element
cout << v.front(); // first element
cout << v.back();  // last element
```

```
v.size();     // number of elements
v.empty();    // true if empty
v.clear();    // remove all elements
```

```
#include <algorithm>

sort(v.begin(), v.end());
```

# Code examples

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {10, 20, 30, 40};

    // Declare iterator
    vector<int>::iterator it;

    // Traverse using iterator
    for(it = nums.begin(); it != nums.end(); ++it) {
        cout << *it << " "; // Dereference to access value
    }
}
```

**Output: 10 20 30 40**

# STL – list in C++

**list** is a **doubly linked list** in STL.

- Fast insertion/deletion at any position: O(1) (if iterator is available)
- No random access (list[0] is not allowed)
- Elements are stored non-contiguously

```
list<int> l;
 l.push_back(10);
 l.push_back(20);
 l.push_front(5);
 for (int x : l)
    cout << x << " ";

Output: 5 10 20
```

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> l = {10, 20, 40};
    auto it = l.begin();
    ++it;               // points to 20
    l.insert(it, 15);
    for (int x : l)
        cout << x << " ";
}
Output: 10, 15, 20, 40
```

# STL – dqueue in C++

deque stands for **Double Ended Queue**.

It is **similar to a vector**, but allows efficient insertion and deletion at both front and back.

```
#include <iostream>
#include <deque>
using namespace std;
int main() {
    deque<int> dq;
    dq.push_back(20);
    dq.push_back(30);
    dq.push_front(10);
    for(int x : dq)
        cout << x << " ";
    return 0;
}
```

**Output: 10, 20, 30**

```
dq.insert(dq.begin()+2, 30);

dq.erase(dq.begin()+1);


for(auto it = dq.begin(); it != dq.end(); it++)
{
    cout << *it << " ";
}
```

# STL – array in C++

std::array is a fixed-size container introduced in C++11.

It is safer and more feature-rich than a normal C-style array..

```
#include <iostream>
#include <array>
using namespace std;

int main()
{
    array<int, 5> arr = {10, 20, 30, 40, 50};

    for(int x : arr)
        cout << x << " ";

    return 0;
}
```

```
auto it = find(arr.begin(), arr.end(), 30);

if(it != arr.end())
    cout << "Found";
else
    cout << "Not Found";


arr.fill(100);

arr.swap(otherArray)
```

# STL – set in C++

A **set** is an STL container that stores unique elements in sorted order.

- Elements are sorted
- Duplicate values are not allowed
- Searching is efficient (O(log n))
- Implemented using a Binary Search Tree

**Interview Example**: Remove Duplicates from an array
**Quick Solution :**

```
#include <iostream>
#include <set>
using namespace std;
 int main()
{     int arr[] = {1,2,3,2,4,1,5};
      set<int> s(arr, arr + 7);
      for(int x : s)    cout << x << " ";
}
```

# STL – multiset in C++

A multiset is similar to a set, but it allows duplicate elements.

- Elements are sorted
- Duplicate values are allowed
- Search, insert, delete: O(log n)
- Implemented using Red-Black Tree

```
#include <set>
multiset<int> ms;

Count Occurrences
cout << ms.count(10);

Delete All Occurrences
ms.erase(10);

Delete Only One Occurrence
auto it = ms.find(10);
ms.erase(it);
```

**Common functions -**

**insert()**
**erase()**
**find()**
**count()**
**lower_bound()**
**upper_bound()**
**equal_range()**
**size()**
**empty()**
**clear()**

# STL – multiset in C++

**Interview Question : Count frequency of an element in an array**
**Quick Solution :**

```
#include <iostream>
#include <unordered_set>
using namespace std;
int main()
{
   int arr[] = {1,2,1,3,2,1};
   unordered_multiset<int> ums(arr, arr + 6);
   cout << "Frequency of 1 = " << ums.count(1);
   return 0;
}
```

# STL – map in C++

A map stores data in key-value pairs.

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<int, string> m;
    m[1] = "One";
    m[2] = "Two";
    m[3] = "Three";
    auto it = m.find(2);
    if (it != m.end())
        cout << "Found: " << it->second;
    else
        cout << "Not Found";
}
```

Output: **Found: Two**

```
if you give -
auto it = m.find(10);

if (it == m.end())
    cout << "Key not found";

Never use
        if(m[10] != 0)
Because it will create m[10] = 0
value
```

Common functions

insert()
find()
erase()
count()
size()
empty()
begin()
end()
lower_bound()
upper_bound()

# STL – multimap in C++

A multimap is similar to a map, but it allows multiple values for the same key.

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    multimap<int, string> mm;
    mm.insert({10, "Pooja"});
    mm.insert({20, "Lokesh"});
    mm.insert({10, "Payal"});
    for(auto p : mm)
    {
        cout << p.first  << " : " << p.second << endl;
    }
    return 0;
}
```

# STL – multimap in C++

A multimap is best when one key needs to store multiple related values, such as:

Department → Employees
Course → Registered Students
Category → Products
Employee ID → Multiple Projects

```
auto range = mm.equal_range(10);
  This returns a pair of iterators:
  range.first   // points to first occurrence of key 10
  range.second  // points just after the last occurrence of key 10

for(auto it = range.first;  it != range.second;  ++it)
{
    cout << it->second << endl;
}
```

# STL – set, unordered set, multiset and unordered multiset in C++

| | **Set** | **Unordered set** | **Multiset** | **Unordered multiset** |
|---|---|---|---|---|
| Unique Elements | Yes | Yes | No ( have duplicates ) | No ( have duplicates ) |
| Sorted | Yes | No | Yes | No |
| DS used | Red-Black Tree | Hash | Red-Black Tree | Hash-table |
| Searching | O(log n) | O(1) | O(log n) | O(1) |

# STL – map, unordered map, multimap and unordered multimap in C++

| | **Map** | **Unordered map** | **Multimap** | **Unordered multimap** |
|---|---|---|---|---|
| Key-value Pair | Yes | Yes | Yes | Yes |
| Duplicate keys allowed | No | No | Yes | Yes |
| Kes Sorted | Yes | No | Yes | No |
| DS used | Red-Black Tree | Hash table | Red-Black Tree | Hash-table |
| Searching | O(log n) | O(1) | O(log n) | O(1) |
| Operator[] supported | Yes | Yes | No | No |
| equal_range() | Less Common | Less | Very useful | Very useful |

# Algorithms

*Algorithms* act on containers.

STL algorithms are pre-built functions that perform common operations (like searching, sorting, counting, reversing, merging, finding, finding max / min ) on containers.

They provide the means by which you will manipulate or extract the contents of containers.

Instead of writing your own logic, you just call required function and it's done.

Purpose: Save time, reduce bugs, and make code more readable.

# Categories of Algorithms

**Sorting & Rearranging** - sort(), reverse(), rotate(), shuffle()
Example: Sorting a playlist by popularity.

**Searching** - find(), binary_search(), count()
Example: Checking if a friend's username exists in your follower list.

**Modifying** - copy(), replace(), remove()
Example: Replacing all "offline" statuses with "online".

**Numeric** - accumulate(), inner_product()

# Code examples

1. **sorting.cpp**

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {5, 2, 8, 1};
    sort(v.begin(), v.end()); // Ascending sort
    for(int x : v) cout << x << " ";
}
```

**Output: 1 2 5 8**

**2. searching.cpp**

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40};
    if(find(v.begin(), v.end(), 30) != v.end())
        cout << "Found 30!";
}
```

**Output: Found 30!**

# Code examples

**3. counting.cpp**

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 3, 2};
    cout << count(v.begin(), v.end(), 2);

    return 0;
}
```

**Output: 3**

**4. accumulate.cpp**

```
#include <iostream>
#include <vector>
#include <numeric>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};
    int sum = accumulate(v.begin(), v.end(), 0);
    cout << "Sum = " << sum;
}
```

**Output: Sum = 60**

# Functors

The STL includes classes that overload the **function call operator()**. Instances of such classes are called function objects or functors.

**Functors** let you pass custom behavior into STL algorithms (like sorting with your own rules).

```
#include <iostream>  // Simple factor
using namespace std;

class Add {

  public:
    // overload function call operator
    // accept two integer arguments
    // return their sum
    int operator() (int a, int b) {
      return a + b;
    }
};
```

```
int main() {

  // create an object of Add class
  Add add;

  // call the add  object
  int sum = add(100, 78);

  cout << "100 + 78 = " << sum;

  return 0;
}
```

# Functor with STL Algorithm

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Greater {
    bool operator()(int a, int b) {
        return a > b; // Custom
comparison
    }
};
```

```
int main() {
    vector<int> v = {1, 4, 2, 8};
    sort(v.begin(), v.end(), Greater()); // Sort descending
    for(int x : v) cout << x << " ";
}
```

# RTTI

RTTI stands for Run Time Type Identification

RTTI enables us to identify type of an object during execution of program

RTTI operators are runtime events for polymorphic classes and compile time events for all other types.

# typeid

- typeid is an operator which returns reference to object of type_info class
- type_info class describes type of object

Use of typeid –

- Polymorphism: Safely handle objects when you don't know their exact type at compile time.
- Type Checking: Compare object types at runtime.
- Safe Casting: Avoid undefined behavior when converting pointers.

# Casting Operators

Explicit conversion is referred as cast

- const cast
- static cast
- dynamic cast
- reinterpret cast

cast operators are sometimes necessary

Explicit cast, allows programmer to momentarily suspend type checking

Syntax

```
cast_name<type>(expression);
```

# casting Operators : const_cast

casts away the constness of its expression

```
#include <iostream>
using namespace std;

void print (char * str) {
    cout << str << endl;
}
```

```
int main () {
const char * c = "sample text";
print ( const_cast<char *> (c) );
return 0;
}
```

# casting Operators : static_cast

- Any conversions which compiler performs implicitly can be made explicit using static_cast
- Warning messages for loss of precision will be turned off.
- Reader, programmer and compiler all are made aware of fact of loss of precision.
- We can think of static_cast as a safe costume change

# casting Operators : static_cast

- Any conversions which compiler performs implicitly can be made explicit using static_cast
- Warning messages for loss of precision will be turned off.
- Reader, programmer and compiler all are made aware of fact of loss of precision.
- We can think of static_cast as a safe costume change

# casting Operators : reinterpret_cast

Complex <double> *pcom;

char *pc = reinterpret_cast < char*>(pcom)

Reinterpret cast performs low level interpretation of bit pattern

Is used to convert any data type to any other data type

The operator provides a conversion between pointers to other pointer types and numbers to pointers and vice versa.

Most dangerous

# casting Operators : dynamic_cast

dynamic_cast can be used only with pointers and references to objects.

Its purpose is to ensure that the result of the type conversion is valid.

dynamic_cast is used for safe runtime type conversion in an inheritance hierarchy.

It is mainly used in polymorphism when a base class pointer/reference points to a derived class object.

# Smart Pointer

Smart pointers in C++ are wrappers around raw pointers that automatically manage memory.

They ensure that dynamically allocated objects are properly deleted when no longer needed, preventing memory leaks and dangling pointers.

Types of smart pointer :  unique, shared, weak

# Unique Pointer  std::unique_ptr

Owns a resource exclusively (only one pointer can manage it).

Automatically deletes the object when it goes out of scope.

Cannot be copied, but can be moved.

Single owner,  Activa key

# Shared Pointer  std::shared_ptr

Multiple pointers can share ownership of the same resource.

Keeps a reference count; deletes the object when the last shared_ptr goes out of scope.

Family members watching Netflix / Prime Video

# Weak Pointer  std::weak_ptr

A non-owning reference to a shared_ptr.

Doesn't increase reference count

Demo File – uniquePointer.cpp

Thank You !!!